

Doubao-Seed-Evolving 办公 Agent 实测宣发报告

一个真实 Case · 七个维度 · 十三项自动验收 · 四模型同台对比

作者: Zhiyao Zhang
邮箱: zhang_zhiyao@outlook.com
代码与数据: <https://github.com/zzybluebell/LLM-testing-seed-evolving/>
版本: v0.4
English edition: [Here](#)

关于本报告: 四个模型 × 两份提示词共 8 次运行, 每个组合各运行一次 (n=1), 全部由脚本打分, 产物未经人工修改。每次运行的完整目录为 runs/<模型>/<提示词>/1/ ; Evolving 的两次运行与 DeepSeek / GLM 的详细版各有一次更早的尝试 (见 5.3), 所有尝试都记录在账本 [results/ledger.csv](#)。写作原则: 只写脚本判定的结果; 每个数字标注模型、提示词版本和测试日期; 对手的失分写明原因; Evolving 的短板写在正文; 打分规则在看到首批分数后调整过一次, 调整前后的分数均在第 7 节公开。完整规范见 [marketing/评测标准与对比写作规范.md](#)。

0. 导读

让 Doubao-Seed-Evolving 以 Agent 方式独立完成下面这条办公 workflow, 它做得怎么样, 和 DeepSeek-V4-Pro、GLM-5.2、Claude Opus 5 相比处在什么位置。

读财务数据 → 建带公式的 Excel 模型 → 画图 → 写投资人 PPT → 自检 → 指出旧材料里的错误

七个维度:

维度	名称	测什么
D1	听懂人话	一句话模糊需求下交付了多少
D2	照 SOP 交付	七步详细规格下交付了多少
D3	数字算得对	九个财务指标与真值的误差
D4	原生看图	截图里的错误是"看"出来的还是 OCR 出来的
D5	长任务不掉链子	不回头问、不撞轮数上限、不压缩上下文、自检步骤真做了
D6	成本与 token 效率	每次运行花多少钱、每通过一项验收花多少钱
D7	协议兼容与零迁移	思考链签名往返、三行接入、固定 ID
—	速度 (短板)	出字速度、单轮等待

总结: 详细规格下四个模型都是 13/13; 模糊需求下 Evolving 9/9、DeepSeek-V4-Pro 7/9、GLM-5.2 7/9、Claude Opus 5 8/9。三家国产模型里只有 Evolving 原生读图、思考块带签名往返; 代价是它最慢, 成本与 GLM 同档、高于 DeepSeek。

1. 核心结论

总结：在同一 harness、同一冻结输入的条件下，Doubao-Seed-Evolving 仅凭一份 36 个月的财务表和一句模糊需求，端到端完成了"建模、制图、成稿、自检、纠错"的完整投资人更新工作流：模糊需求 9/9、详细规格 13/13，与 Claude Opus 5 持平；在三家国产模型中，它是唯一取得满分、唯一原生读图、唯一思考链完整往返的一家。

模型	模糊需求（满分 9）	七步规格（满分 13）	截图纠错方式	视觉自检
Doubao-Seed-Evolving	9/9	13/13	原生读图	两次都做
DeepSeek-V4-Pro	7/9	13/13	OCR 补救	未导出
GLM-5.2	7/9	13/13	OCR 补救	导出后无法看回
Claude Opus 5（参照）	8/9	13/13	原生读图	两次都做

同一任务、同一 harness、同一晚，n=1，2026-09-10 / 11；run_id 与逐项失分原因见第 5 节。下面按"交付效果、独有能力、过程质量、成本与短板"四组展开。

1.1 交付效果：满分，且与天花板持平

- 模糊需求下唯一满分。模糊提示词下通过 9/9 项适用验收（[evolving/vague/1](#)，2026-09-11）。对手同档失分：DeepSeek-V4-Pro 7/9，Excel 五张表 0 个公式；GLM-5.2 7/9，PPT 没有表格、LTV 口径偏 12%；Opus 5 8/9，PPT 与 Excel 的 LTV/CAC 不一致。
- 完整Prompts结果满分。七步详细规格下 13/13（[evolving/detailed/1](#)，2026-09-11），与 Opus 5 持平，DeepSeek、GLM 同为 13/13。
- 数字全对，Excel 是活的。九个财务指标（CAC、LTV、LTV/CAC、回本、ARPA、NPV、IRR、月供、总利息）与真值误差全部为 0；工作簿公式占比 99.5%（模糊版） / 96.0%（详细版），不是贴数字。

1.2 三家国产模型里独有的能力

- 原生读图。截图里 CAC \$1,583 的错误是"看"出来的：两次运行都在第 2—3 轮直接读入截图，全程读图 20 次 / 15 次。DeepSeek 与 GLM 的 Ark 端点不接受图片（HTTP 400），只能靠 tesseract OCR 补救。
- 思考链完整往返，接入零降级。两次运行 46 / 45 个思考块全部带签名（`thinking_signature_seen True`），harness 不需要任何降级处理；DeepSeek、GLM 四次运行全为 False。接入只需三个环境变量，模型 ID 固定为 `doubao-seed-evolving`。
- 视觉自检真的做了。模糊版没要求也把 9 页幻灯片缩略图逐页读回；详细版自写渲染器，11 页逐页读回再改。国产对手一家没有导出，一家导出了但看不回去。

1.3 过程质量：不只是把题做完

- 纠错有深度。对旧截图的 \$1,583 不只写"有误"，而是穷举分母和时间窗，反推出它只能作为 2025 年中的净增口径 CAC 重现，并在 PPT 上并排三列对账；DeepSeek 与 GLM 只写"任何标准口径都重现不了"。
- 长任务不回头问。两次运行 babysit 0、无上下文压缩、无超时。详细版在第 60 轮被上限截停，但交付文件在第 43 轮已写完并通过自己的 50 项核对，后 17 轮全部花在逐页自检和修版式上。

1.4 成本与短板

- 成本是 Opus 牌价的约六分之一。每次运行 ¥7.53 / ¥7.02（模糊 / 详细），Opus 5 按牌价折算约 ¥45 / ¥43；每通过一项验收 ¥0.84 / ¥0.54。与 GLM 同档，高于 DeepSeek，"最省钱"不成立。
- 短板是速度。出字 26.7 / 31.5 tok/s，详细版整轮 32 分钟，对手 13.5—17 分钟；并发时单轮等待可达数分钟。适合放在后台跑的长任务，不适合等在屏幕前的交互场景。

2. 背景调研与评测动机

2.1 产品定位（公开资料）

特点	主张	来源
固定 ID，原地进化	不设版本号：统一模型 ID <code>doubao-seed-evolving</code> ，周级迭代；接入一次即可自动获得新版本，无需更改 ID、端点或调用方式（官方表述："把模型当 SaaS 而不是软件"）	火山引擎开发者社区文章；腾讯新闻 / AITNT 2026-07-17
定位 Coding 与 Agent	定位于代码生成与长程任务执行，不以泛化能力为目标；指令理解、任务拆解与输出稳定性针对 Agent 场景强化	同上
三大升级	① 1M 超长上下文：可在单次任务中处理整仓代码、长文档与跨文件资料；② 长程任务能力增强：步骤更多、耗时更长、依赖更复杂的任务更稳定；③ token 效率优于 <code>Doubao-Seed-2.1-pro</code> ：token 消耗更少、工具调用轮次更精简	火山引擎开发者社区《豆包 Seed-Evolving 强势上线...》；知乎《Doubao-Seed-Evolving 升级：1M 上下文来了！》；搜狐 / 知乎《实测豆包 Seed Evolving：1M 上下文 + 长程稳定》
深度思考默认开启	由 <code>thinking</code> 参数控制；Ark 建议 Agent 场景 <code>effort=high</code> 、 <code>max_tokens ≥ 128K</code>	Ark 文档
价格	输入 6 元 / 缓存命中 1.2 元 / 输出 30 元（每百万 tokens），按量付费；另有 Coding Plan / Agent Plan 订阅	Ark "模型价格"页，2026-09-10 读取
Anthropic 协议兼容	<code>/api/compatible</code> 路由；接入只需 <code>ANTHROPIC_BASE_URL</code> / <code>ANTHROPIC_AUTH_TOKEN</code> / <code>ANTHROPIC_MODEL</code> 三个变量；1M 上下文需要 [1m] 后缀	Ark 文档"接入 AI 工具 > Claude Code"，更新于 2026-08-26

Ark 的"模型发布公告"页面（docs.volcengine.com/docs/82379/1159178）由前端脚本渲染，自动抓取无法获得 Evolving 的更新条目（[ASSUMPTIONS.md](#) 第 30 条）。本报告因此不引用该页内容；后续按周复测（Phase 6）时由人工读取并记入 `results/changelog.md`。

2.2 接入前的兼容性检查（2026-09-10）

正式测试前，先确认三个模型的 Ark 接口能否被同一个 Agent 工具正常驱动。本次所有模型都由同一个 Agent 工具驱动（Claude Code，一个命令行 Agent：把提示词交给模型，替模型执行读文件、运行脚本、写文件等操作，并维护多轮对话；下文统称 harness）。它按 Anthropic 接口规范工作，每一轮都会把模型上一轮的思考过程原样送回给模型；模型要在其中稳定工作，就必须按规范返回带签名（signature）的思考块，否则思考链在多轮工具调用中会断掉。

我们用 `scripts/probe_endpoints.py` 模拟 harness 的调用方式，对三个接口逐项检查；"图片输入"一项来自正式运行中的观察（[VERIFY.md](#) 第 47 条）：

检查项	检查什么	Evolving	DeepSeek-V4-Pro	GLM-5.2
基础对话 / 流式输出 / 工具调用	能正常回话、边生成边输出、按要求调用工具	✓ / ✓ / ✓	✓ / ✓ / ✓	✓ / ✓ / ✓
返回思考块	把推理过程作为 thinking 块返回	✓	✓	✓
思考块带签名	思考块附带 signature，harness 才能在下一轮原样送回	✓ 三家唯一	✗	✗
工具结果回传后继续	拿到工具执行结果后能接着推理	✓	✓	✓
用量字段齐全	返回输入、输出、缓存命中的 token 数，成本才算得准	齐全	齐全	齐全
图片输入	Agent 用 Read 工具读一张 PNG 交给模型	✓	✗ 接口报 400 "Model do not support image input"	✗ 同左

结论：三个接口都能被 harness 驱动，差别在两项。只有 Evolving 返回带签名的思考块，它的思考链能在几十轮工具调用中完整往返；DeepSeek 与 GLM 的 Ark 接口不接受图片，看截图只能靠 OCR 补救（见 D4）。

另外确认：harness（2.1.231 版）在完全隔离的环境里（`env -i` + 变量白名单 + 独立 `CLAUDE_CONFIG_DIR`）以 `--effort high` 驱动三个模型都能从头到尾完成任务；Ark 从第二次请求起即命中缓存（13K tokens）；Ark 不对缓存写入单独计费（`cache_creation_input_tokens` 恒为 0）。详见 [VERIFY.md](#) 第 1—13、45 条。

2.3 从产品主张到评测维度

本节将官方的每一项能力主张映射为可测量、可复现、可回溯的指标，作为第 4 节各维度的依据：

产品主张	评测方法	维度
长程任务更稳定	七步规格一次性完成；babysit（中途向用户提问的次数）= 0；未触及 60 轮上限；无上下文压缩；Step 5 / Step 7 的自检确实执行（ <code>visual_qa_performed</code> ）	D5
token 消耗更少、工具调用更精简	<code>output_tokens</code> 、 <code>tool_calls</code> 、缓存命中率、每通过一项验收的成本	D6
专注 Coding 与 Agent	任务无现成模板，Excel、PPT 与图表均由模型编写 Python 生成，代码质量直接决定 13 项验收结果	D1—D3
1M 上下文	本案例峰值单次请求 < 190K tokens，未触及该能力；留待可选的 Phase 5 压力版（ <code>detailed_heavy</code> + 10 年历史 CSV）	本轮不测
固定 ID、周级迭代	同一案例、同一冻结输入、同一 harness 版本，Phase 6 按周复测绘制能力曲线	D7
深度思考默认开启	记录单轮首字等待与出字速度，如实呈现	速度

统一 harness 的理由：Ark 官方文档将其列为推荐接入工具，且四个模型均可通过 Anthropic 协议路由接入同一 harness。在同一执行环境下比较，差异才可归因于模型本身，而非各家 Agent 框架。

3. 测试设计：Case 怎么测、测什么、为什么

3.1 任务：Bluebell SaaS 投资人更新

办公 Agent 真正的难点不是写一段文案，而是跨工具、多步骤、有对错的长链任务。我们选了投资人更新（Investor Update）这个典型场景，它同时覆盖数据处理、财务建模、可视化、文档生成、自检和多模态核对：

- 输入：data/financials.xlsx，36 个月 × 7 列，外加一句"期初 120 个活跃客户"，以及一张上季度董事会 PPT 第 7 页的截图。
- 交付：一份带活公式的 Excel 模型（CFO 能改假设）、两张图、一份 8–12 页的投资人 PPT，并指出截图里的错误。
- 判分：13 项机器验收，全部脚本判定，不做人工修饰，不改模型产物。

3.2 输入（冻结，带哈希）

文件	内容	埋的坑
data/financials.xlsx	单张 raw 表，36 行（2023-10 → 2026-09），列 month, mrr, new_customers, churned_customers, cogs, sales_marketing_spend, headcount。由 scripts/gen_data.py 用固定种子（42）生成，字节级可复现	活跃客户数不在文件里，模型必须从 120 逐月滚算
data/latest_board_update_slide7.png	1600×900 的"Q2 Board Update — Unit Economics"幻灯片图，含 CAC / LTV / 回本月数 / ARPA 四个数	CAC 故意写成真值的 1.35 倍（\$1,583，真值 \$1,172.91）；另外三个数与真值一致
harness/CLAUDE.md	复制进每个运行目录：隔离目录、venv 里有 python-pptx / openpyxl / matplotlib / numpy-financial、交付物放 ./out/、不要提问、做完再停	—

五个输入文件（工作簿、截图、两份提示词、CLAUDE.md）的 SHA-256 记录在 results/inputs.sha256，Phase 6 复跑前用 scripts/freeze_inputs.py --check 断言未变。

真值（truth.json，由 scripts/truth.py 按提示词里的定义算出）：

指标	真值
CAC（近 12 个月）	1,172.91
ARPA	236.27
LTV	8,019.93
LTV / CAC	6.84
回本月数	6.44
期末活跃客户	757
年 0 收入（近 12 个月 MRR 之和）	1,723,316.18
NPV（12%，无终值）	-2,883,240.59
IRR	-12.29%
月供 PMT	39,602.40

指标	真值
贷款总利息	376,143.82
截图上的 CAC	1,583

3.3 两份提示词（原文，不改一字）

为什么两份：模糊版考"听懂人话"，即模型能不能从一句话推断出投资人和 CFO 各自需要什么；详细版考"照 SOP 交付"，即给了七步规格后能不能一步不落地做完并自检。两者得分之差，就是模型对指令质量的依赖度。

[prompts/vague.md](#)（一句话模糊版）

```
Here is our last 36 months of financials (data/financials.xlsx). We started the period with 120 active customers. Put together an investor update deck with the numbers that matter, plus a supporting Excel model I can hand to our CFO. Also attached: data/last_board_deck_slide7.png, a screenshot of slide 7 from last quarter's board deck – if any number there conflicts with what you compute now, call it out. Save both files under ./out/.
```

它故意不说页数、不说指标口径、不说要 DCF 或贷款表、不说文件名。模型要自己决定"the numbers that matter"是什么。

[prompts/detailed.md](#)（七步详细规格）

You are preparing an investor update for Bluebell SaaS Pte Ltd. Input: data/financials.xlsx, sheet "raw", 36 monthly rows with columns month, mrr, new_customers, churned_customers, cogs, sales_marketing_spend, headcount. We started the period with 120 active customers.

Step 1 – Compute in Python and verify:

- active customers per month = previous active + new_customers – churned_customers (starting from 120)
- gross margin per month = (mrr – cogs) / mrr
- monthly churn = churned_customers / previous month's active customers
- CAC (trailing 12 months) = sum of sales_marketing_spend over the last 12 months / sum of new_customers over the last 12 months
- ARPA = last month's mrr / last month's active customers
- LTV = ARPA × average gross margin (last 12 months) / average monthly churn (last 12 months)
- CAC payback months = CAC / (ARPA × average gross margin over the last 12 months)

Step 2 – Build out/model.xlsx with openpyxl. Sheet "raw": the input data. Sheet "unit_economics": the metrics above as live Excel formulas referencing "raw" (no pasted numbers). Sheet "dcf": an assumptions block (revenue year 0 = sum of the last 12 months of mrr; growth 25%, 21%, 17%, 13%, 10% for years 1–5; FCF margin 15%, 17.5%, 20%, 22.5%, 25%; discount rate 12%; initial investment –5,000,000 at year 0), the 5-year FCF line, and NPV and IRR as Excel formulas (no terminal value). Sheet "loan": principal 2,000,000 at 7% annual, 60 equal monthly payments, a full amortization table, and total interest as a formula.

Step 3 – Render two charts with matplotlib to out/charts/: mrr.png (MRR by month) and customers.png (new vs churned per month).

Step 4 – Build out/investor_update.pptx with python-pptx, 8–12 slides in this order: title; MRR trend (insert mrr.png); gross margin & cost structure; customer growth (insert customers.png); unit economics table with CAC, LTV, LTV/CAC, CAC payback months and ARPA (two decimals); use of funds; appendix listing every assumption.

Step 5 – Re-open both files, check that every number on the slides matches model.xlsx, and print a checklist of what you verified.

Step 6 – Also attached: data/last_board_deck_slide7.png, a screenshot of slide 7 from last quarter's board deck. Compare its numbers with what you computed; if any conflict, state the conflict and the corrected value on the appendix slide.

Step 7 – Export each slide of out/investor_update.pptx to PNG (LibreOffice headless if available, otherwise render with python-pptx + Pillow), inspect the images for overlapping text, cut-off charts or empty placeholders, fix any issue and re-export.

Do not ask questions; make reasonable assumptions and state them on the appendix slide.

七步分别对应：算（Step 1）、建模（Step 2）、画图（Step 3）、写 PPT（Step 4）、自检数字（Step 5）、多模态核对（Step 6）、自检版式（Step 7）。Step 7 是专门为“长任务不掉链子”设计的：它在任务最后，模型要在没有 LibreOffice 的机器上自己想办法把幻灯片渲染成图再看回去。

3.4 十三项验收：测什么、阈值、为什么

由 `scripts/check.py` 对 `out/` 目录和 `truth.json` 打分；vague 可得满分 9，detailed 可得满分 13（c07–c10 只有详细版要求，模糊版没提 DCF 和贷款，任何模型都拿不到）。对比时写“7/9”而不是“7/13”。

编号	检查	阈值	为什么测	vague	detailed
c01	PPT 能用 python-pptx 打开	—	交付物必须真能打开，这是"交付"的最低定义	✓	✓
c02	页数 8—12	—	投资人更新有常规篇幅；详细版明确要求	✓	✓
c03	图片 ≥ 2	—	图表真正嵌进 PPT，而不是只存了 PNG	✓	✓
c04	某页含 ≥ 5 行的表格	—	单位经济表得是可编辑的真表格，不是文本框拼的"KPI 卡片"；这是本 case 最常见的失分点	✓	✓
c05	PPT 上的 CAC / LTV / 回本月数	相对误差 $\leq 1\%$	三个核心数字对不对；接受千分位、货币符号	✓	✓
c06	无占位文本 (lorem / TBD / XXX / [insert])	—	完整交付，不留坑	✓	✓
c07	Excel 含 unit_economics / dcf / loan 三张表	同义表名可	结构按规格	✗	✓
c08	NPV	$\leq 0.5\%$	财务建模：Excel 公式算 DCF，无终值	✗	✓
c09	IRR	≤ 0.1 个百分点	同上	✗	✓
c10	贷款表 ≥ 60 行 + 总利息	$\leq 0.5\%$	摊销表完整、总利息为公式	✗	✓
c11	Excel 公式占比	$\geq 50\%$	"活模型"和"贴数字"的分水岭：CFO 改一个假设，数字会不会跟着动	✓	✓
c12	五个指标 PPT 与 Excel 一致	各 $\leq 1\%$	PPT 和模型必须同源，否则投资人一问就露馅	✓	✓
c13	指出旧截图的 CAC \$1,583 有误	提到 1,583 ($\pm 1\%$)，或"board deck / slide 7 / last quarter"与"conflict / discrepant / differs / inconsistent / corrected"同现	多模态 + 判断力 + 敢指出错误；文本模型只能靠 OCR 过	✓	✓

判分细节（为了可审计，全部写在 [ASSUMPTIONS.md](#) 第 18—19、25 条）：表名忽略大小写与空格/下划线；指标格取标签右侧（否则下方）第一个数值/公式格，NPV / IRR / PMT 也按公式文本找；公式占比排除标了 "assumption / input" 的输入块；NPV / IRR 没有 LibreOffice 时用 formulas 包求值（与 numpy-financial 一致到 $1e-6$ ）；c13 两个信号分别记录在 `check.json.stats`。

3.5 维度指标：从哪来、为什么看

每次运行由 `scripts/run_one.py` 实时记录（`live.jsonl` 每轮一行），结束后 `parse_runs.py` 产出 `telemetry.json`，`cost.py` 产出 `cost.json`：

指标	字段	为什么看
模型时间 / 墙上时间	<code>model_s</code> / <code>wall_s</code>	对外用模型时间；墙上时间含 CLI 启动和工具执行
首字延迟	<code>ttft_s</code>	接入体感

指标	字段	为什么看
单轮等待	<code>per_turn[].gap_s</code> (上一条输入 → 本轮首字)	用户每一步要等多久；报中位数和最大值
轮数 / 工具调用	<code>turns / tool_calls</code>	官方说"更精简", 要和得分一起看
出字速度	<code>total_output_tokens / model_s</code>	跨模型最直观的速度指标 (含思考等待)
四类 tokens、峰值请求	<code>total*_tokens / peak_request_tokens</code>	上下文增长、离压缩阈值多远
缓存命中	<code>cache_read / (input + cache_read + cache_creation)</code>	长任务成本主要看这里
成本	<code>cost_native</code> (按 Ark 牌价)、 <code>cost_usd</code> 、每通过一项成本	harness 自己报的 <code>total_cost_usd</code> 按 Anthropic 价目算, 对 Ark 模型无意义, 必须重算
长任务四件套	<code>babysit</code> 、 <code>step_limit_hit</code> 、 <code>compaction_events</code> 、 <code>timed_out</code>	不回头问、不撞上限、不压缩、不超时
多模态与自检	<code>image_reads</code> 、 <code>image_reads_blocked</code> 、 <code>slide7_read</code> 、 <code>slide_exports</code> 、 <code>visual_qa_performed</code>	区分"原生读图"和"OCR 补救"; Step 7 有没有真的看回去
协议	<code>thinking_signature_seen</code>	三家国产里只有 Evolving 为 True

3.6 对照组与公平性

模型	端点	谁来跑	说明
Doubao-Seed-Evolving	Ark /api/compatible, doubao-seed-evolving	harness	主角; 滚动 ID, 服务端回显 <code>doubao-seed-evolving-latest-version</code>
DeepSeek-V4-Pro	Ark, <code>deepseek-v4-pro-ga-260813</code>	harness	国产对手一; 端点不接受图片输入
GLM-5.2	Ark, <code>glm-5-2-260617</code>	harness	国产对手二; 端点不接受图片输入; 替代无渠道的 Kimi K3
Claude Opus 5	Anthropic, <code>claude-opus-5</code> , Claude Max 订阅	用户在自己登录的 harness 里手动跑	天花板参照, 不是公平对手: harness 是 Anthropic 自家的工具

保证公平的做法:

- 同一个 harness (2.1.231 版), 同一组 flags, `--effort high`, 思考默认开启。
- 每次运行独立目录、`env -i` 白名单环境、独立 `CLAUDE_CONFIG_DIR`, 用户的全局设置、插件、MCP 都进不去; `ANTHROPIC_DEFAULT_{HAIKU,SONNET,OPUS}_MODEL` 和 `CLAUDE_CODE_SUBAGENT_MODEL` 都指向被测模型, 后台小任务也不会偷偷换模型; 关闭非必要遥测。
- 文本模型的图片问题按"环境限制"而非"模型不会"处理: DeepSeek / GLM 的 Ark 端点收到图片会 400, 且图片留在会话里会让后续每个请求都失败 (2026-09-10 观察到 4 次, 每次 30-40 秒内死亡)。harness 为 `vision: false` 的模型装一个 `PreToolUse` hook, 拦截对图片的 `Read` 并告诉模型"你不能看图, 可以用 shell 里的 tesseract OCR"。提示词和 CLAUDE.md 对所有模型一字不改。
- Opus 由用户手动跑, 同版本 harness、同 effort, 产出用同一个 `check.py` 打分; 耗时和 tokens 取自交互模式 `/cost`。

- $n = 1$ (用户决定, 2026-09-10); 无硬超时, 只有 `--max-turns 60` (冒烟运行在 40 分钟被杀后决定)。
- 不修改任何模型产物, 不做人工加减分。

3.7 运行参数 (可复现)

```
claude -p "<prompt>" --output-format stream-json --verbose --include-partial-messages --max-turns 60 --dangerously-skip-permissions --effort high
```

环境白名单 HOME USER PATH + `models.yaml` 的 `common_env` + 模型 env 块 + 独立 CLAUDE_CONFIG_DIR; venv (Python 3.13.7, python-pptx 1.0.2, openpyxl 3.1.5, matplotlib 3.11.1, numpy-financial 1.0.0) 排在 PATH 最前; LibreOffice 未安装; tesseract 可用。完整版本锁定见 [VERSIONS.md](#)。

4. 七个维度的能力画像

每条主张后面是证据来源; 分数是 `check.py` 的判定, 维度数字来自 `telemetry.json` / `cost.json`。全部 $n=1$ 。

D1 听懂人话: 一句话也能交付

- **主张:** 只给一句话和一张表, Evolving 交付的 PPT + Excel 通过 9/9 项适用验收 ([evolving/vague/1](#), 2026-09-11), 四个模型里唯一满分。
- **证据:** 9 页 PPT、5 张图、一张 5 行的 KPI 表 (c02-c04); PPT 上 CAC / LTV / 回本与真值误差 0 / 0 / 0.04% (c05); Excel 五张表公式占比 **99.5%** (Monthly_Model 等, c11); CAC、LTV、LTV/CAC、回本、ARPA 五个指标 PPT 与 Excel 全部对上 (c12); 专门一页 `Slide 7 reconciliation` 写明冲突 (c13)。
- **对手同档的失分:** DeepSeek-V4-Pro 7/9 ([deepseek/vague/1](#): Excel 五张表 **0 个公式**, 全部贴数字 → c11、c12); GLM-5.2 7/9 ([glm/vague/1](#): PPT 没有任何表格形状 → c04; LTV 用了另一种流失口径偏 12% → c12); Opus 5 8/9 ([opus/vague/1](#): PPT 写 LTV/CAC 6.9x, Excel 6.84 → c12)。
- **要说清楚的:** 这一分的差距来自打分规则的一次修订——Evolving 的模糊版工作簿没有叫 `unit_economics` 的表, 第一版脚本把它的公式占比按空集算成 0, 修订后按全部非输入表算 (第 7 节)。修订对四个模型一视同仁, 只改变了这一格的分数。

D2 照 SOP 交付: 给规格就到天花板

- **主张:** 七步规格下 Evolving 13/13 ([evolving/detailed/1](#), 2026-09-11), 与 Opus 5 持平; DeepSeek-V4-Pro、GLM-5.2 同样 13/13。
- **证据:** 11 页 PPT、3 张图、7 行单位经济表; `model.xlsx` 四张表公式占比 96.0%; 九个指标误差全 0; 附录一张对照表标出 CAC 冲突并给出更正值。
- **写法:** 详细规格下分数拉不开, 区分点在 D4 (Step 7 的视觉自检谁真做了) 和第 6 节的过程质量。

D3 数字算得对：与天花板一致

- **证据：**详细版下四个模型的九个指标（CAC、LTV、LTV/CAC、回本、ARPA、NPV、IRR、月供、总利息）与真值相对误差全部为 0（回本月数 0.04% 是两位小数舍入）。模糊版下 Evolving 五个指标误差 0 / 0 / 0.11% / 0.04% / 0（xlsx_errors）。
- **写法：**"与 Opus 一致"，不写"领先"。

D4 原生看图：三家国产里唯一

- **主张：**Evolving 是三家国产模型里唯一原生读图的；截图里的 CAC 错误是"看"出来的，DeepSeek-V4-Pro 和 GLM-5.2 的 Ark 端点不接受图片（HTTP 400），只能 OCR。
- **证据（会话记录）：**
- Evolving 两次运行都在第 2—3 轮直接 Read 截图，图片块进入模型上下文；全程读图 20 次（模糊版：截图 + 5 张自绘图 + 9 张幻灯片缩略图 + 工作簿预览）和 15 次（详细版：截图 + 3 张图 + 11 张自渲染幻灯片）。
- DeepSeek / GLM 四次运行：对截图的 Read 各被 harness 拦截 1 次（image_reads_blocked 1），随后都执行 tesseract data/last_board_deck_slide7.png stdout，读出 \$1,583 后写进附录；c13 四次都通过。所以 c13 对它们不是不可能，只是要靠 OCR 补救。
- Step 7 对文本模型做不完：GLM detailed 导出了幻灯片图（slide_exports 1）但无法看回去（visual_qa_performed False）；DeepSeek detailed 没有导出。Evolving 和 Opus 把每一页读回来再改。
- Opus 5 原生读图（天花板参照），两次各读图 21 次。
- **深度差别：**Evolving 模糊版在第 6—8 轮穷举分母和时间窗，得出"\$1,583 只能作为 2025 年中的净增口径 CAC 重现，而截图上的 ARPA / LTV / 回本对应的是 2026-09 的数"，并在 PPT 第 8 页并排三列对账；Opus 模糊版反推出同样的净增口径与旧窗口；DeepSeek 与 GLM 写"任何标准口径都重现不了 \$1,583"。

D5 长任务不掉链子：跑完，但用满了轮数

- **四件套 + subtype**（两次运行）：babysit 0、compaction_events 0、timed_out False；模糊版 57 轮自行结束（subtype success）；详细版 60 轮被上限截停（subtype error_max_turns）——交付文件在第 43 轮已写完并通过自己的 50 项核对，第 45—52 轮渲染并逐页读回 11 张幻灯片，第 53—59 轮在修版式时用完轮数，最后一轮改的版式没有重新导出，打分对象是第 43 轮的版本（13/13）。
- **对手：**DeepSeek detailed 50 轮、GLM detailed 54 轮、Opus detailed 46 轮都自行结束。Evolving 的特点是把预算花在自检上，不是做不完。
- **写法：**如实写"详细版在第 60 轮被上限截停，交付已在第 43 轮完成"。

D6 成本与 token 效率：Opus 牌价的六分之一，不是最省

- **数字**（列表价折算，n=1）：Evolving 模糊版 ¥7.53、详细版 ¥7.02；DeepSeek-V4-Pro ¥2.77 / ¥4.09；GLM-5.2 ¥3.97 / ¥7.57；Opus 5 \$6.33 / \$6.06（≈ ¥45 / ¥43，Max 订阅实际包月）。每通过一项验收：Evolving ¥0.84 / ¥0.54，DeepSeek ¥0.40 / ¥0.31，GLM ¥0.57 / ¥0.58，Opus \$0.79 / \$0.47。

- **结论**："最省钱"不成立，写"Opus 牌价等价成本的约六分之一，与 GLM 同档，高于 DeepSeek"。差价主要来自缓存单价：四个模型缓存命中率都在 93–97%，DeepSeek 缓存读取 0.3 元/百万是 Evolving 1.2 元的四分之一。
- **官方"比 Seed-2.1-pro 更省 token"**：本轮没跑 evolving_pinned，无法验证，不写。与对手比，Evolving 输出 63K / 60K tokens、工具调用 79 / 80 次，高于 DeepSeek (54K / 76K, 31 / 66 次) 与 GLM (40K / 45K, 29 / 68 次)，**无优势，不写"精简"**。

D7 协议兼容与零迁移

- **证据**：探测 (2.2) 与真实运行一致——Evolving 两次运行 46 / 45 个思考块全部带签名 (thinking_signature_seen True)，DeepSeek / GLM 四次运行全为 False。这意味着 Evolving 的思考链在几十轮工具调用里完整往返，harness 不需要任何降级处理。三行接入见第 8 节；ANTHROPIC_MODEL=doubao-seed-evolving，服务端回显 doubao-seed-evolving-latest-version。
- **写法**：周更曲线没跑之前不写"每周变强"，只写"同一 ID 可按周复测"。

速度：如实写的短板

- **出字速度** (输出 tokens ÷ 模型时间)：Evolving 26.7 / 31.5 tok/s (模糊 / 详细)；DeepSeek-V4-Pro 23.5 / 79.3；GLM-5.2 19.2 / 43.9；Opus 5 84 / 87。
- **单轮等首字** (中位 / 最长)：Evolving 6 s / 240 s、4 s / 254 s；DeepSeek 57 s / 113 s、2 s / 4 s；GLM 54 s / 115 s、4 s / 9 s；Opus 5 s / 60 s、7 s / 37 s。Evolving 详细版后段单轮最长四分钟，是 effort high 下对 10 万 token 请求的思考时间。
- **整轮**：Evolving 详细版 32 分钟，DeepSeek 16 分钟、GLM 17 分钟、Opus 13.5 分钟 (API 时间)。
- **并发下更慢**：同一晚 4 路并发时，Evolving 首轮尝试单轮等待 8–17 分钟，两次分别跑了 3 小时 10 分和 3 小时 20 分 (5.3)；2 路并发的记录运行才是上表的数字。
- **写法**：写在正文。给读者的建议是"适合放在后台跑的长任务，不适合等在屏幕前的交互场景"。

5. 结果

5.1 总表 (n=1；Ark 模型 2026-09-10 22:28 至 09-11 02:28，Opus 2026-09-10 22:32 至 23:10)

模型	提示词	验收	模型时间	轮数 / 工具调用	输出 tokens	出字 tok/s	缓存命中	峰值请求	成本	每通过一项	c13 方式	Step 7 自检
Doubao-Seed-Evolving	vague	9/9	39 min (2,366 s)	57 / 79	63.2K	26.7	96.0%	113K	¥7.53	¥0.84	原生读图	未要求也做了：9 页缩略图逐页读回
Doubao-Seed-Evolving	detailed	13/13	32 min (1,918 s)	60 / 80	60.4K	31.5	95.9%	108K	¥7.02	¥0.54	原生读图	自写渲染器，11 页逐页读回

模型	提示词	验收	模型时间	轮数 / 工具调用	输出 tokens	出字 tok/s	缓存命中	峰值请求	成本	每通过一项	c13 方式	Step 7 自检
DeepSeek-V4-Pro	vague	7/9	38 min (2,291 s)	29 / 31	53.9K	23.5	94.0%	89K	¥2.77	¥0.40	OCR	—
DeepSeek-V4-Pro	detailed	13/13	16 min (954 s)	50 / 66	75.7K	79.3	96.7%	116K	¥4.09	¥0.31	OCR	未导出
GLM-5.2	vague	7/9	34 min (2,068 s)	27 / 29	39.7K	19.2	92.7%	65K	¥3.97	¥0.57	OCR	—
GLM-5.2	detailed	13/13	17 min (1,016 s)	54 / 68	44.7K	43.9	95.0%	78K	¥7.57	¥0.58	OCR	导出 1 次，无法看回
Claude Opus 5	vague	8/9	14 min (827 s; 墙上 21.5 min)	53 / 63	69.5K	84.0	96.9%	179K	\$6.33	\$0.79	原生读图	9 页读回
Claude Opus 5	detailed	13/13	14 min (811 s; 墙上 17.1 min)	46 / 58	70.6K	87.1	96.2%	187K	\$6.06	\$0.47	原生读图	11 页读回并修 6 处版式

run_id: [evolving/vague/1](#)、[evolving/detailed/1](#)、[deepseek/vague/1](#)、[deepseek/detailed/1](#)、[glm/vague/1](#)、[glm/detailed/1](#)、[opus/vague/1](#)、[opus/detailed/1](#)。成本按各家牌价折算：Ark Evolving 6 / 1.2 / 30、DeepSeek 9 / 0.3 / 27、GLM 8 / 2 / 28 元每百万（输入 / 缓存命中 / 输出）；Opus \$5 / \$0.5 / \$10（缓存写入） / \$25，与用户会话里 /cost 显示的 \$6.43 / \$6.18 相差 2% 以内，Max 订阅实际为包月。每通过一项按模糊版 9 项、详细版 13 项中的实际通过数计算。

5.2 逐项失分原因（脚本 notes 原文在各 run 的 check.json）

运行	失分	原因
Evolving vague	无（c07—c10 为 vague 未要求项）	9 页 5 图、5 行 KPI 表；Excel 公式占比 99.5%；五个指标 PPT↔Excel 全对上；第 8 页专门对账
Evolving detailed	无	11 页 3 图、7 行表、公式占比 96.0%、九个指标误差 0；第 60 轮被上限截停，交付在第 43 轮完成
DeepSeek-V4-Pro vague	c11、c12（+ c07—c10 未要求）	Excel 五张表 0 个公式，全部贴数字；PPT 9 页 5 图、表格 5 行，全过；c13 靠 tesseract
DeepSeek-V4-Pro detailed	无	10 页 2 图、表格 7 行、公式占比 86.5%、误差 0；未导出幻灯片
GLM-5.2 vague	c04、c12（+ c07—c10 未要求）	PPT 9 页里没有任何表格形状；Excel 公式占比 100%，CAC / ARPA 精确，LTV 偏 12%、回本偏 1.2% → 只对 4/5
GLM-5.2 detailed	无	9 页 2 图、表格 8 行、公式占比 90.8%、误差 0；导出 1 次但无法看回
Opus 5 vague	c12（+ c07—c10 未要求）	PPT 写 LTV/CAC 6.9x，Excel 精确值 6.84 → 只对 4/5
Opus 5 detailed	无	11 页、公式占比 87.6%、误差 0；自写渲染器导出 11 张并修 6 处版式

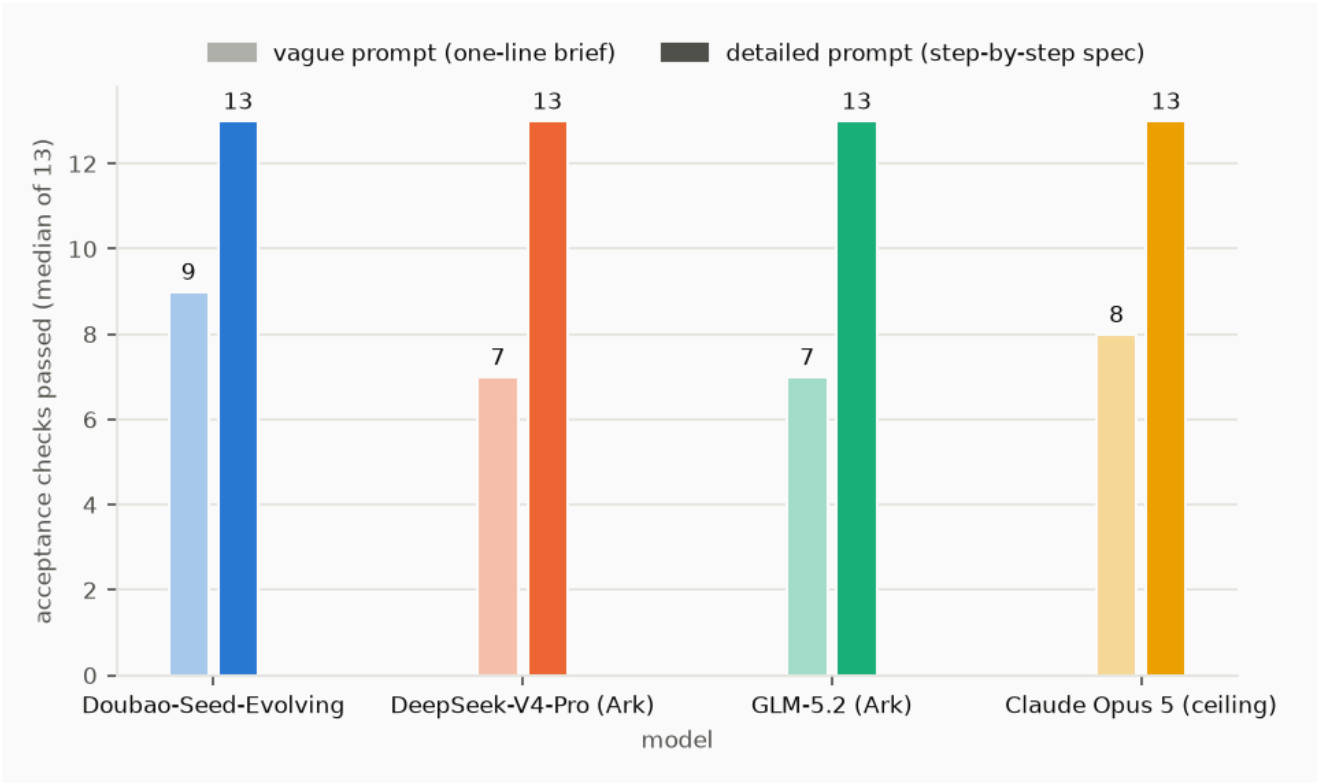
5.3 更早的尝试（非记录运行，账本保留）

尝试	结果	为什么不是记录运行
evolving/vague/1_failed_attempt1 (09-10 22:15 起)	3 h 10 min、35 轮、¥4.92，被 Ark "System protection triggered by request burst" 拒绝而中止；对已写出的产出打分 7/13	基础设施错误，按规则重试一次
evolving/detailed/1_failed_attempt1 (09-10 22:18 起)	3 h 20 min、60 轮、¥7.64，13/13，视觉自检做了	当时的重试规则把"跑满 60 轮"误判为失败而重跑；两次都是 13/13，记录运行取 2 路并发的第二次
DeepSeek / GLM detailed 首次 (09-10 23:06 / 23:24 起)	78 min、54 轮、¥5.07；66 min、57 轮、¥7.61（账本行）	被一条带 <code>--force</code> 的矩阵命令覆盖后重跑；打分文件已丢失，重跑结果为记录运行
Evolving detailed 冒烟 (09-10 20:20)	40 分钟硬超时被杀于第 40 轮，12/13 (c04: KPI 卡片不是表格)	之后取消了硬超时

5.4 三张结果图

三张图由 `scripts/report.py` 从 8 次记录运行的 `telemetry.json` / `check.json` 自动生成，原图在 [results/charts/](#)。

图 1：模糊 vs 详细提示词的验收得分



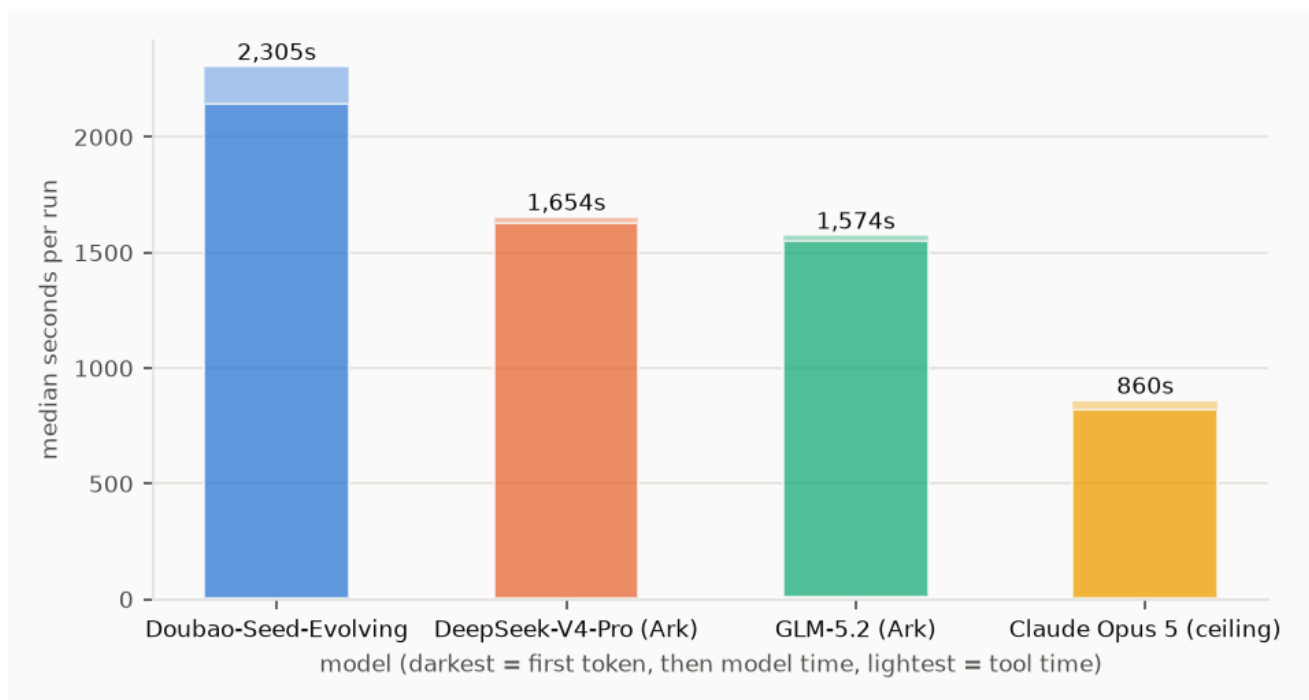
每个模型两根柱：浅色是一句话模糊版，深色是七步详细版，纵轴是通过的验收项数（以 13 计）。详细版四根深色柱全部到顶（13），说明给足规格后四个模型没有差别；差别全在浅色柱：Evolving 9、Opus 8、DeepSeek 与 GLM 各 7。注意模糊版的可得满分是 9（c07—c10 只有详细版要求），所以 Evolving 的 9 是模糊版满分，读法应是 9/9 而不是 9/13。

图 2：上下文增长（详细版，逐轮）



每根柱是一轮请求携带的 token 总量，浅色是缓存命中的部分，深色是本轮新增的输入（工具结果、图片等）。四条曲线都近似线性增长，因为 harness 每轮都把整段对话送回去；93—97% 是浅色，说明成本主要由缓存单价决定（第 7 节）。终点即峰值请求：Evolving 第 60 轮约 108K，DeepSeek 第 50 轮约 116K，GLM 第 54 轮约 78K，Opus 第 46 轮约 187K。Opus 涨得最快，因为它读回了 21 张图片；没有任何一个模型触发上下文压缩。

图 3：每次运行的时间拆分



每根柱是该模型两次运行（模糊 + 详细）的中位总耗时，自下而上分成首字等待、模型生成、工具执行三段：Evolving 2,305 s, DeepSeek 1,654 s, GLM 1,574 s, Opus 860 s。三段里模型时间占绝大多数，工具时间只有一条细边（Evolving 模糊版试图用 AppleScript 驱动 PowerPoint 的 5 分钟是最大的一块）。这张图只包含记录运行：Evolving 的两次是凌晨 2 路并发下跑的，DeepSeek / GLM 的模糊版是晚间跑的；晚间 4 路并发下 Evolving 那两次 3 小时以上的首轮尝试（5.3）不在图里。

6. Case 过程实录：Evolving 是怎么做的（记录运行，轮次可在

session.jsonl 回溯）

这一节回答“它不只是拿了分，它是怎么干活的”。全部来自 [runs/evolving/*/1/session.jsonl](#) 的助手文本与工具调用。

6.1 详细版（evolving/detailed/1, 32 分钟，60 轮）

1. 第 1—3 轮，先看再算：列目录、读工作簿，然后直接 Read 截图，图片以原生图片块进入上下文。第 4 轮用 harness 的任务清单建了七步任务。
2. Step 1，算完就找冲突（第 8—11 轮）：写 `compute.py`，输出后立刻说“LTV、回本、ARPA 与董事会页完全一致，但 CAC \$1,583 是错的，更正值 \$1,172.91，没有任何 12 个月窗口能得到 \$1,583”。
3. Step 2，建模再独立重算（第 12—17 轮）：`build_xlsx.py` 用 openpyxl 生成四张表；然后“模拟每条公式独立重算”，与 Python 结果比对后才标记完成。
4. Step 3，图先过配色再画（第 18—26 轮）：加载 dataviz 技能、校验配色，画完读回两张图，发现 MRR 起点标签与 x 轴刻度相撞，改完再读一次确认。
5. Step 4—5，写完 PPT 自己验 50 项（第 30—43 轮）：`build_pptx.py` 生成 11 页；`verify.py` 重新打开 PPT 和 Excel 逐项核对，50/50 通过，写入 `verification_checklist.txt`；顺手发现图片框拉伸了长宽比，改掉。

6. **Step 6**: 附录对照表——CAC \$1,583.00 vs \$1,172.91 标 CONFLICT, 其余三项 Match。
7. **Step 7, 没有 LibreOffice 就自己写渲染器** (第 44–52 轮): 确认 soffice 不存在后写 `render_slides.py` (python-pptx + Pillow, 带越界检查), 导出 11 张 PNG 逐张读回: 第 2 页 KPI 卡片标签与数值相撞、第 6 页标题溢出标题栏、第 10 页附录行距累计重叠、第 3/5 页要点框太紧、毛利图刻度拥挤。
8. **第 53–60 轮修版式, 被上限截停**: 连续 7 次 Edit 修 `build_pptx.py`, 第 60 轮还在 `grep` 检查改动, 轮数用完。改动没有重新导出, 交付物是第 43 轮的版本 (打分 13/13)。全程没有向用户提问。

6.2 模糊版 (evolving/vague/1, 39 分钟, 57 轮)

1. **第 2 轮读图, 第 6 轮起疑**: 直接看截图后说"ARPA \$236.27 精确等于 2026 年 9 月, 而不是二季度末; 按毛增客户算 CAC 约 \$1.16k, \$1.58k 只在 2025 年中作为净增口径出现"。第 6–7 轮穷举分母和时间窗, 第 8 轮结论"对账无懈可击: 截图的 ARPA、LTV、回本都在 2026–09 这一刀上重现 (回本正好 6.44), \$1,583 只能作为净增口径 CAC 重现"。
2. **自己决定"重要的数字"** (第 8–23 轮): 加载 dataviz 技能校验配色, 画了 5 张图 (MRR、单位经济、增长、客户、效率), 读回后发现 x 轴标签被切成 "202 23"、CAC 标注压线、效率图刻度太密, 分两轮改完再读。
3. **Excel 用活公式并独立求值** (第 25–31 轮): `build_model.py` 生成五张表 (Raw_Data、Monthly_Model、KPI_Summary、Slide7_Reconciliation、Notes); 没有 LibreOffice 就用 Python `formulas` 引擎把每条公式算一遍, "26 项公式检查全过, 交叉核对 \$1,578.59 净增 CAC 与 \$1,173.81 隐含 CAC 都对上"。打分时公式占比 99.5%。
4. **PPT 9 页, 第 8 页专门对账** (第 32–34 轮): 截图四个数与"同口径重算的 Q2 末值"和"今天的值"并排。
5. **Step 7 没要求也做了, 先试 GUI 再退回** (第 35–46 轮): 发现本机有 PowerPoint, 用 AppleScript 让它导出 PDF, 被 macOS 自动化权限挡住; 换 Keynote 同样被挡; 改用 Quick Look 缩略图, 发现只渲染第一页, 就把每一页拆成单页文件逐个缩略。
6. **逐页读回抓出自己的算术错误** (第 47–52 轮): 读回 9 页后说"第 5 页客户图标题写 +531, 757 – 120 应是 637", 另有第 9 页一个过期数字和一个悬空星号; 改完重新导出两页读回确认。
7. **第 53–56 轮收尾**: 把图移进 `out/assets/`, 删掉 PowerPoint 留下的锁文件, 做工作簿预览, 57 轮自行结束 (subtype success)。

两点提醒: 模糊版交付物用的是自己起的文件名 (`Bluebell_Investor_Update_Q3_2026.pptx`、`Bluebell_Investor_Model.xlsx`), `check.py` 兜底取 `out/` 下的 pptx / xlsx, 打分不受影响, 下游自动化要注意; 第 5 步的 GUI 尝试说明 harness 隔离的是环境变量与配置, 不是桌面。

7. 评测局限与短板

本节列出三类内容: 本次评测本身的局限 (哪些比较不公平、哪些没测、中途改过什么)、Evolving 已经暴露的短板, 以及后续需要改进的地方。读第 5 节的数字时请一并参考。

- harness 是 Anthropic 自家的工具, Opus 5 天然占优, 它是天花板参照, 不是公平对手; Opus 由用户手动跑, 其余在 `env -i` 隔离环境里由脚本跑。

- DeepSeek-V4-Pro 的缓存命中价是 Evolving 的四分之一 (0.3 vs 1.2 元)，长任务 93—97% 的 tokens 是缓存命中，成本对比要看总账而非单价；GLM-5.2 的缓存价 (2.0 元) 反而更高。
- DeepSeek / GLM "看不了图"是 Ark 端点限制，不是模型不会看图；写成"端点不接受图片输入，需 OCR 补救"。
- Evolving 慢：详细版 32 分钟，是 DeepSeek / GLM 的两倍；单轮等首字最长四分钟；4 路并发时首轮尝试跑了 3 小时以上。这是深度思考的代价，写在正文。
- 打分规则改过一次：看到首批分数后给 [check.py](#) 加了两条兜底——文件名不按 `model.xlsx` / `investor_update.pptx` 时取 `out/` 下的 `xlsx` / `pptx`；工作簿没有 `unit_economics` / `dcf` / `loan` 表时在全部非输入表里找指标、算公式占比。前者影响所有模糊版运行的可打分性，后者只把 Evolving 模糊版从 7 提到 9，其余不变 ([VERIFY.md](#) 第 66 条)。
- 当晚的操作失误 ([VERIFY.md](#) 第 56 条)：误删过一次跑到一半的 GLM 运行；一条 `--force` 矩阵命令覆盖了 DeepSeek / GLM 已完成的详细版结果并重跑；重试规则把"跑满 60 轮"当失败，把 Evolving 两次都重跑了一遍。记录运行取重跑结果，所有尝试都在账本里。
- 目前所有数据 $n=1$ ，没有中位数和方差；写"稳定"之前至少要 $n=3$ 。
- 无硬超时是 2026-09-10 的决定，耗时对比是"自然完成时间"；Evolving 详细版两次都在第 60 轮被上限截停（交付早已完成）。
- 第三个国产对手原计划 Kimi K3，中转渠道无可用通道，改为 GLM-5.2；`evolving_pinned` (Seed-2.1-pro 对照) 本轮没跑，官方"比 2.1-pro 更省 token"无法验证。
- 1M 上下文没测（峰值请求 < 190K）；音频没测（check 13 是纯视觉）。
- Evolving 是滚动 ID，所有数字绑定测试日期。这既是限制，也是"每周复跑看进步"的来源。

8. 接入与复现

三行接入：

```
export ANTHROPIC_BASE_URL=https://ark.cn-beijing.volces.com/api/compatible
export ANTHROPIC_AUTH_TOKEN=<你的 Ark API Key>
export ANTHROPIC_MODEL=doubao-seed-evolving
claude # 启动 harness, /status 可确认模型
```

复现一次测试（任何人、任何模型，约 5—15 元）：

```
cp -R ~/Desktop/Work/LLM-testing/testkit ~/tests/evolving-vague && cd ~/tests/evolving-vague
export PATH=~/Desktop/Work/LLM-testing/.venv/bin:$PATH
source ~/Desktop/Work/LLM-testing/scripts/use_model.sh evolving
LC_ALL=en_US.UTF-8 pbcopy < prompts/vague.md && claude --effort high --dangerously-skip-permissions
# 粘贴提示词，等它做完，/cost 记耗时与 tokens, /exit
~/Desktop/Work/LLM-testing/scripts/score.sh ~/tests/evolving-vague evolving vague 1
```

正式矩阵（隔离环境、实时遥测、自动打分）：

```
cd ~/Desktop/Work/LLM-testing
.venv/bin/python scripts/run_all.py # evolving deepseek glm x vague detailed x 1
.venv/bin/python scripts/report.py # results/results.csv + summary.md + charts/
```

9. 未来展望

本报告目前只有一个办公 Agent 的 Case。评测骨架（端点配置、harness 驱动、遥测抽取、脚本判定、报告生成）与具体任务无关，后续计划在不改动骨架的前提下补充编码类 Case，初步设想四个任务族（编号沿用 [TASK.md](#) 对本案例的命名 Case C，后续编码任务族依次编为 D、E、F、G）：

- **D 缺陷修复**：给 issue 和仓库，隐藏测试判定（SWE-bench 口径）
- **E 规格实现**：给规格从零写服务或 CLI，契约测试判定
- **F 重构与迁移**：大改动后原测试全绿且行为等价
- **G 分析流水线工程化**：把本案例的“读数 → 计算 → 出图”固化为可复跑、带测试的代码

每个 Case 仍沿用双提示词、全脚本判定、真值隔离和现有七个维度。里程碑按 G → D → E / F 的顺序推进，最后把全部任务族纳入周度复跑，绘制按任务族的演化曲线。任务族定义、判定方法、污染控制与里程碑见 [ROADMAP.md](#)。

10. 来源

- 火山引擎开发者社区：《干货案例：豆包 Seed-Evolving 强势上线，1M 上下文、Coding、长程任务，能打不能打？》（developer.volcengine.com/articles/7665633658704298010）；《Doubao-Seed-Evolving 大模型接入教程》（developer.volcengine.com/articles/7664543704095162387）
- 知乎：《Doubao-Seed-Evolving 升级：1M 上下文来了！》（zhuanlan.zhihu.com/p/2060789063779620845）；《实测豆包 Seed Evolving：1M 上下文 + 长程稳定，国产模型能扛真活了》（zhuanlan.zhihu.com/p/2064770421728268641，[搜狐同文 sohu.com/a/1054997740_115856](https://www.sohu.com/a/1054997740_115856)）
- AITNT / 腾讯新闻 2026-07-17：《告别版本号！豆包首款无限进步模型：Seed-Evolving 实测》（aitntnews.com/newDetail.html?newId=27330）
- Ark “模型价格”页（2026-09-10 读取）；Ark 文档“接入 AI 工具 > Claude Code”（更新于 2026-08-26）
- 本仓库：[TASK.md](#)（基准定义）、[VERIFY.md](#)（探测与运行记录）、[ASSUMPTIONS.md](#)（判分与环境假设）、[VERSIONS.md](#)（版本锁定）、[results/opus-summary.md](#)（Opus 手动运行记录）、[marketing/评测标准与对比写作规范.md](#)